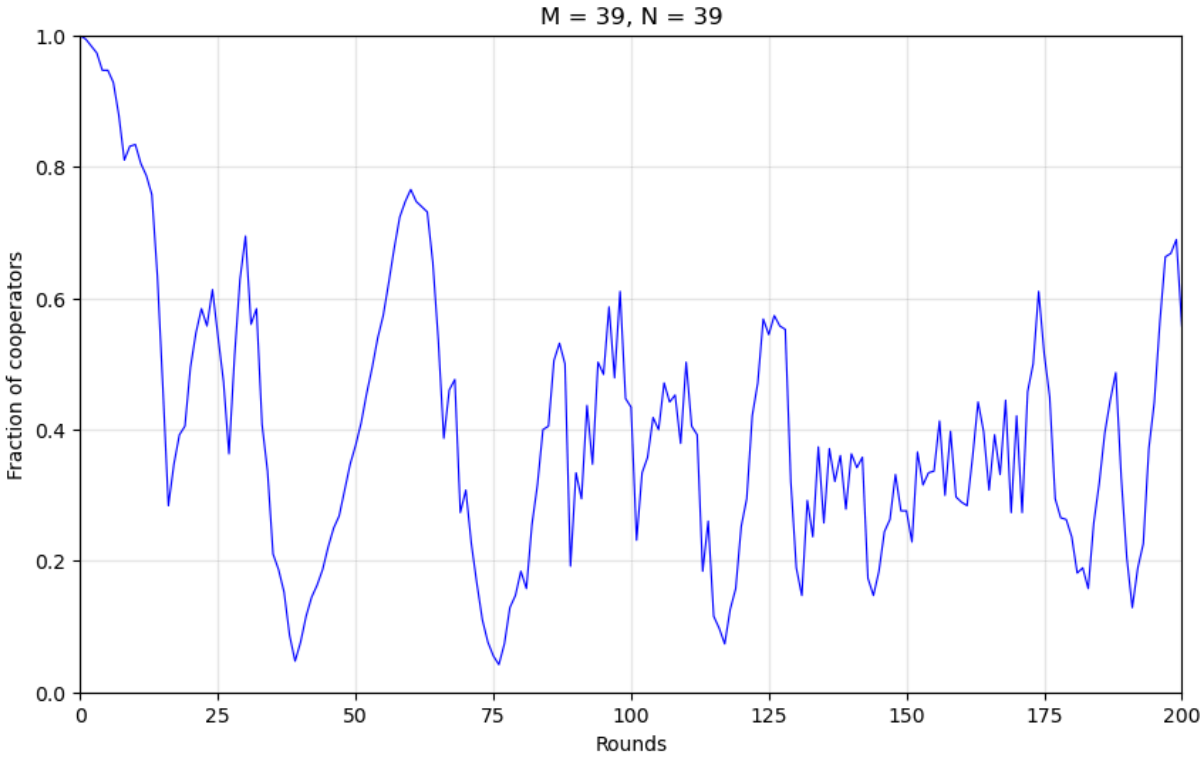
In [6]:
```
"""
Saul Brauns, CMOR 220, Fall semester, Project 9
EvolutionaryGameTheory.ipynb

This program simulates a game theory problem. The game involves the prisoner
At first, there is one defector and the rest are cooperators, then each neig
The matrix is a torus and wraps around its sides. So the left edge borders t


Last Modified: 12/4/25
"""

import numpy as np
import matplotlib.pyplot as plt

# defines the matrix that takes A as its input - this is the matrix we will
def extend_board(A):
# inputs - A (matrix which is functioning as our game board)
# Output - A extended into a torus.

    M, N = A.shape

    # Create a torusified (I don't know if this is a word) board
    A_ext = np.zeros((M + 2, N + 2))

    # Place the original board A in the middle of the torus board A_ext
    A_ext[1:M+1, 1:N+1] = A

    # Top row is old bottom row
    A_ext[0, 1:N+1] = A[M-1, :]
    # Bottom row is old top row
    A_ext[M+1, 1:N+1] = A[0, :]
    # Left column is old right column
    A_ext[1:M+1, 0] = A[:, N-1]
    # Right column is old left column
    A_ext[1:M+1, N+1] = A[:, 0]

    # Above did not deal with any corners. This fills in the four corners. T
    # top right from bottom left, bottom left from top right, and bottom rig
    A_ext[0, 0] = A[M-1, N-1]
    A_ext[0, N+1] = A[M-1, 0]
    A_ext[M+1, 0] = A[0, N-1]
    A_ext[M+1, N+1] = A[0, 0]

    return A_ext


# uses our A_ext to find a score for a prisoner's dilemma scenario
def score(A, b):
# inputs - A (matrix with cooperator and defectors filled in represented by
# outputs - S (a new board with the score that each operator in each square

    M, N = A.shape
```

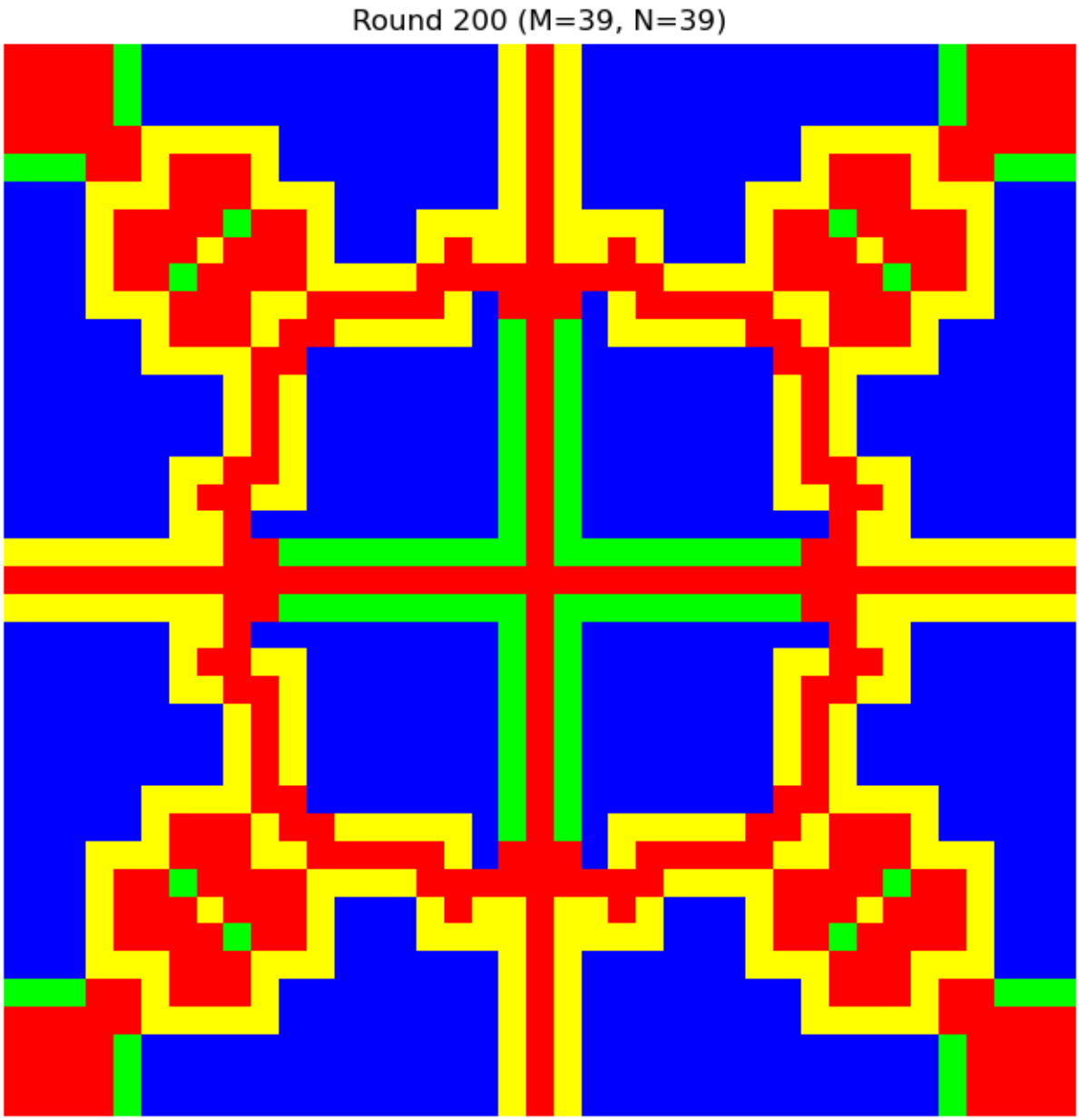
```
    # Call the prior function to use torus magic on A
    A_ext = extend_board(A)

    # Initialize our output score matrix
    S = np.zeros((M, N))

    # Calculate score for each square
    for i in range(M):
        for j in range(N):
            current = A_ext[i+1, j+1]

            total_score = 0

            # Play against all 8 neighbors and self
            for di in range(3):
                for dj in range(3):
                    neighbor = A_ext[i + di, j + dj]

                    # Calculate points for current player in different scena
                    # scenarios: coop vs. coop, coop vs. defec, defec vs coo
                    if current == 1:
                        if neighbor == 1:
                            total_score += 1
                        else:
                            total_score += 0
                    else:
                        if neighbor == 1:
                            total_score += b
                        else:
                            total_score += 0

            S[i, j] = total_score

    return S


# New board generation based on neighbor's highest score. Change to that sty
def advance(S, A):
# Inputs: S (score matrix) and A (game board)
# outputs: An (new modified game board with neighbor score adoption strategy

    M, N = A.shape

    # torus these boards S and A
    S_ext = extend_board(S)
    A_ext = extend_board(A)

    # Initialize next round's board
    An = np.zeros((M, N))

    # For every square, analyze neighbor's scores to find the highest one
    for i in range(M):
        for j in range(N):
            neighborhood_scores = S_ext[i:i+3, j:j+3]

            # Find the index of the maximum score using np.argmax and np.unr
```

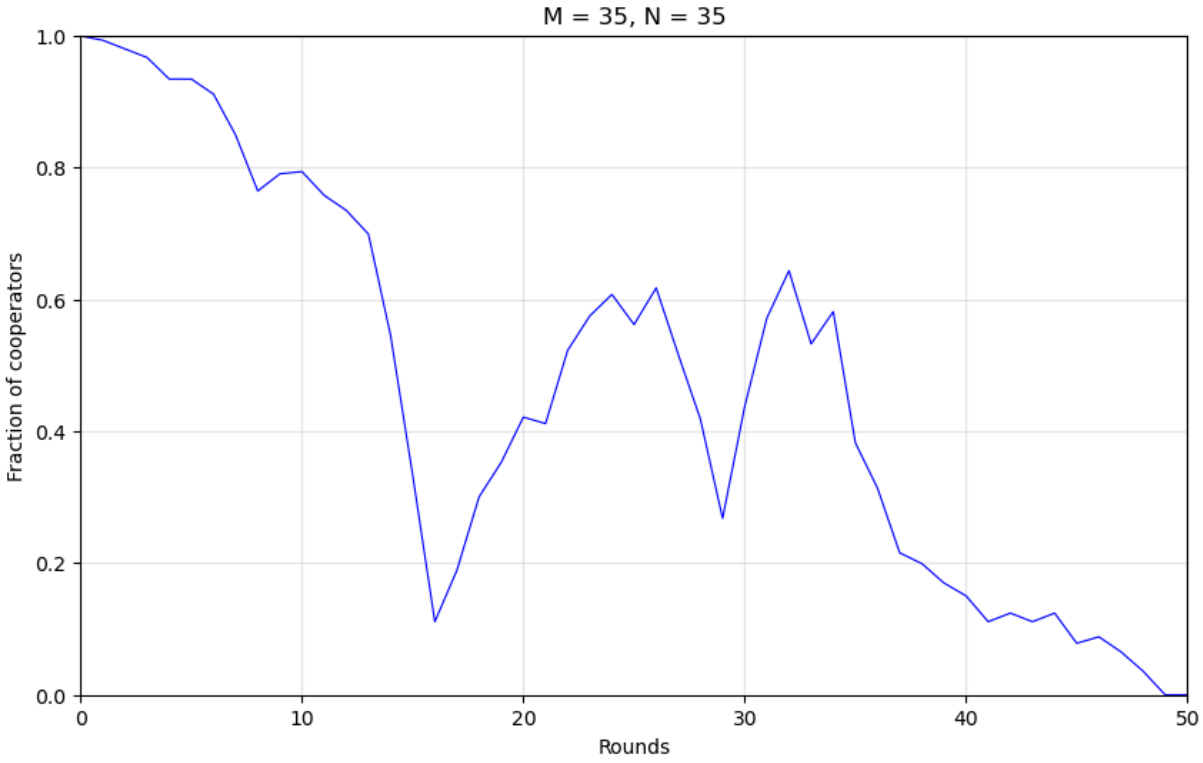
```
            (row, col) = np.unravel_index(np.argmax(neighborhood_scores), ne

            An[i, j] = A_ext[i + row, j + col]

    return An


# Colors the board in based on each boxes status
def evodisp(A, An):
# inputs - A (game baord) and An (modified game baord with neighbor score ac
# outputs - D (3D array which stores color values in addition to game board

    M, N = A.shape

    # Initialize 3D array
    D = np.zeros((M, N, 3))

    for i in range(M):
        for j in range(N):
            if A[i, j] == 1 and An[i, j] == 1:
                # if C stays C then blue
                D[i, j, :] = [0, 0, 1]
            elif A[i, j] == 0 and An[i, j] == 0:
                # if D stays D then red
                D[i, j, :] = [1, 0, 0]
            elif A[i, j] == 1 and An[i, j] == 0:
                # if C changes to D then yellow
                D[i, j, :] = [1, 1, 0]
            else:
                # if D changes to C then green
                D[i, j, :] = [0, 1, 0]

    return D


# Runs the evolutionary game
def evo(M, N, b, rnd):
# inputs - M (rows), N (columns), b (defec points won when against cooperato
# outputs - A (the final game board), fractions (coop fractions from each ro

    A = np.ones((M, N))
    mid_i = M // 2
    mid_j = N // 2
    A[mid_i, mid_j] = 0

    # Track fraction of cooperators over time
    fractions = np.zeros(rnd + 1)
    fractions[0] = np.sum(A) / (M * N)

    # Store display array for later visualization
    D = None

    # Play the game for rnd specified number of rounds
    for r in range(rnd):
        # Score the board and store it in S
        S = score(A, b)
```

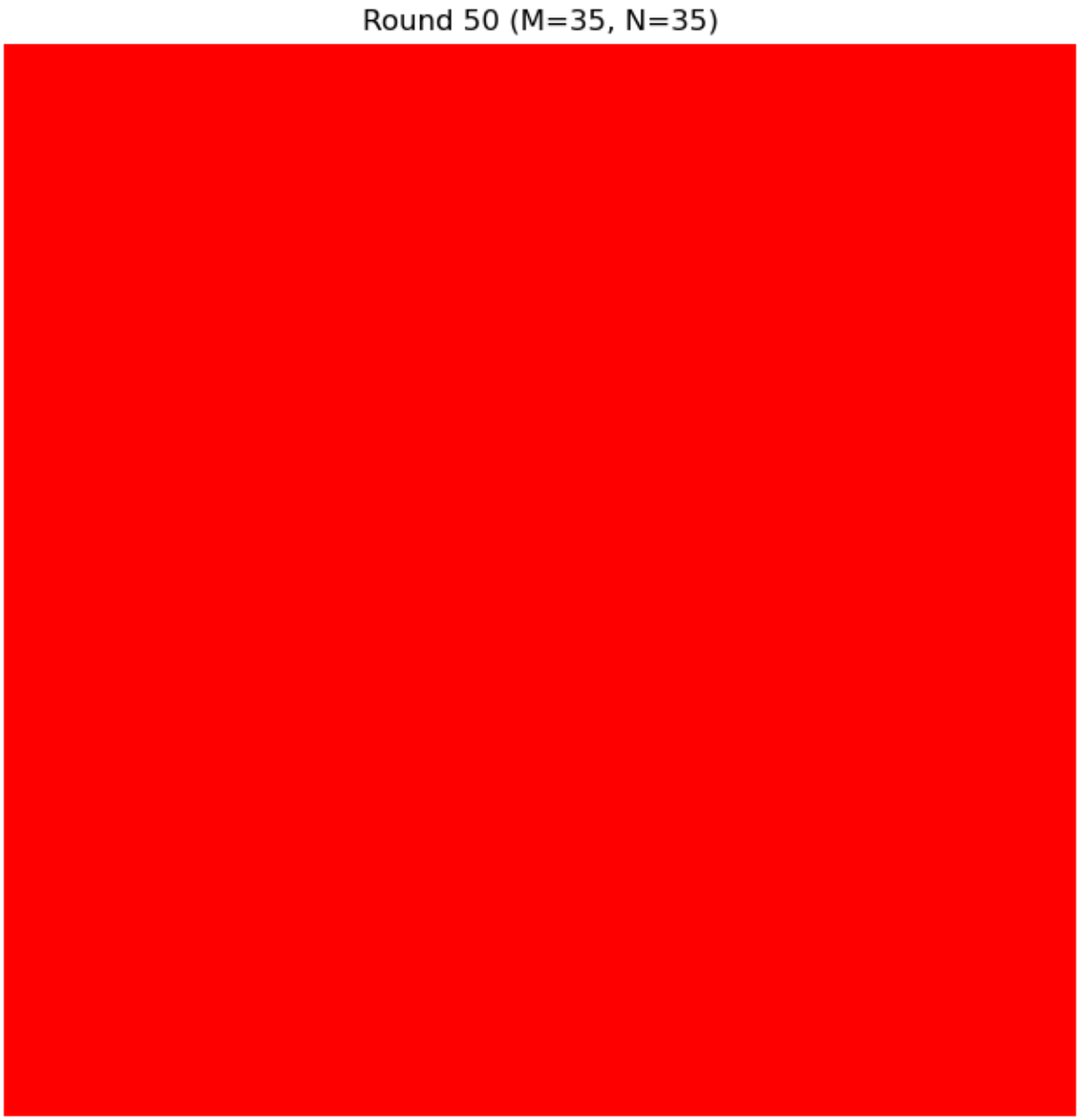
```
        # Advance to the next round
        An = advance(S, A)

        # Create the colorful visualization
        D = evodisp(A, An)

        fractions[r + 1] = np.sum(An) / (M * N)

        A = An.copy()

    return A, fractions, D


# Set the b parameter which is cell's rate of defect
b = 1.9

# first plot
M, N, rnd = 39, 39, 200
A_final, fractions, D_final = evo(M, N, b, rnd)

# Create figure for fraction of cooperators
plt.figure(figsize=(10, 6))
plt.plot(range(rnd + 1), fractions, 'b-', linewidth=0.8)
plt.xlabel('Rounds')
plt.ylabel('Fraction of cooperators')
plt.title(f'M = {M}, N = {N}')
plt.xlim([0, rnd])
plt.ylim([0, 1])
plt.grid(True, alpha=0.3)
plt.savefig('plot1_fraction.png', dpi=150, bbox_inches='tight')
plt.show()

# second plot
plt.figure(figsize=(8, 8))
plt.imshow(D_final)
plt.title(f'Round {rnd} (M={M}, N={N})')
plt.axis('off')
plt.savefig('plot2_board.png', dpi=150, bbox_inches='tight')
plt.show()

# third plot
M, N, rnd = 35, 35, 50
A_final_35, fractions_35, D_final_35 = evo(M, N, b, rnd)

# Fraction of cooperators
plt.figure(figsize=(10, 6))
plt.plot(range(rnd + 1), fractions_35, 'b-', linewidth=0.8)
plt.xlabel('Rounds')
plt.ylabel('Fraction of cooperators')
plt.title(f'M = {M}, N = {N}')
plt.xlim([0, rnd])
plt.ylim([0, 1])
plt.grid(True, alpha=0.3)
plt.savefig('plot3_fraction.png', dpi=150, bbox_inches='tight')
plt.show()
```

```
# Final game board
plt.figure(figsize=(8, 8))
plt.imshow(D_final_35)
plt.title(f'Round {rnd} (M={M}, N={N})')
plt.axis('off')
plt.savefig('plot3_board.png', dpi=150, bbox_inches='tight')
plt.show()

# Analysis for Question 3
print(f"\nFor M=N=35, rnd=50:")
print(f"  Final fraction of cooperators: {fractions_35[-1]:.4f}")
print(f"  Final fraction of defectors: {1 - fractions_35[-1]:.4f}")

print(f"\nFor M=N=39, rnd=200:")
print(f"  Final fraction of cooperators: {fractions[-1]:.4f}")
print(f"  Final fraction of defectors: {1 - fractions[-1]:.4f}")

print(f"\n Questions Answers:")
print(f"  This means that this game ended in all defectors, whereas the prev
print(f"  Nothing will change with more rounds because the defectors cannot
```




Round 200 (M=39, N=39)


M = 35, N = 35
Fraction of cooperators
Rounds
1.0
0.8
0.6
0.4
0.2
0.0
0
10
20
30
40
50




```
For M=N=35, rnd=50:
  Final fraction of cooperators: 0.0000
  Final fraction of defectors: 1.0000

For M=N=39, rnd=200:
  Final fraction of cooperators: 0.5575
  Final fraction of defectors: 0.4425

 Questions Answers:
  This means that this game ended in all defectors, whereas the previous onl
y had a 0.4425 fraction of defectors.
  Nothing will change with more rounds because the defectors cannot adopt an
y style form their neighbors because everyone is a defector.
```

In [ ]: